MLSF Project
Meowdoku and SAT Solving

October 2026

Project overview

The project builds a SAT solver and uses it to solve Meowdoku. In a Meowdoku grid, each cell belongs to one colored region. Cats must be placed so that each region contains exactly one cat, each row and column contains at most one cat, and no two cats touch, including diagonally. A SAT problem asks whether there is an assignment of true or false to Boolean variables that makes a formula true. By assigning one variable to each cell, the game rules can be written as a formula whose satisfying assignments correspond to legal cat placements.

The work forms a sequence. First, a grid is converted into clauses in DIMACS CNF format, the standard text representation used here for SAT instances. Next, a DPLL solver reads those clauses and searches for a satisfying assignment. A satisfying assignment can then be translated back into cat coordinates and displayed on the original grid. Later questions change how the solver chooses its search branches, measure the effects, and specify the final command-line interface. Each stage uses the output of the preceding stage.

1. **Question 1 — Model and generate grids.** Define the Boolean variable for each cell; express the four game rules as CNF clauses; write a converter from the required grid format to DIMACS; and generate grids that have at least one solution by construction.

2. **Question 2 — Read and solve CNF.** Parse DIMACS into a Python formula, implement the status-only and model-producing versions of DPLL, validate them on the supplied instances, and discuss their performance.

3. **Question 3 — Present a game solution.** Connect the grid converter to the SAT solver. Convert a satisfying assignment back into cell coordinates, write the required region-by-region and marked-grid output, and check that the placement obeys the game rules.

4. **Question 4 — Improve branch selection.** Implement and compare at least four DPLL branching heuristics. The selection must include a literal-based rule, a rule that uses clause sizes, and a dynamic rule based on the current simplified formula.

5. **Question 5 — Measure the heuristics.** Instrument the solvers, compare their search effort and run time on varied SAT and UNSAT instances, and analyze whether the extra work of choosing branches pays off.

6. **Question 6 — Provide the final solver interface.** Offer both the basic `dpll` mode and an `optimized` mode through `sat.py`, using the prescribed SAT/UNSAT and optional model output format.

1 Question 1: Encoding and generating Meowdoku grids

File	Description
`image_grid.txt`	The board of Figure 1 in the grid-file format of the assignment (`GRID`, `ZONES`, then one line of region labels per row). Input of the converter.
`meowdoku_to_dimacs.py`	Converter from a grid file to a DIMACS CNF file: `read_grid`, `encode_grid` and `write_dimacs`.
`generate_grid.py`	Generator of square boards with a planted solution: `cat_columns` plants the cats, `generate_grid` grows the regions, and `write_grid` writes the grid file.

1.1 The board and its variables

A Meowdoku grid has N rows, M columns, and K disjoint colored regions. A legal placement has at most one cat in each row and column, exactly one cat in each region, and no two cats in cells that touch along an edge or corner. Figure 1 is the single running example in this report; "the board" below always refers to it. It has $N = M = K = 6$, and its six cats show a complete legal placement. The SAT input uses the colored regions; the cats of Figure 1 illustrate a solution rather than fixed clues in the input file.

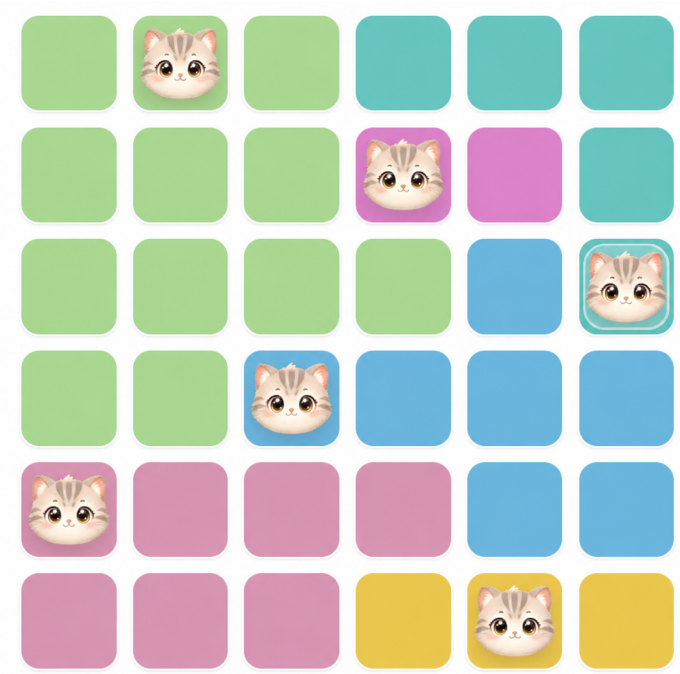


Figure 1: Example of a 6×6 Meowdoku board. Colors define the six regions, and the six cats show a complete solution.

We name the green, turquoise, pink, blue, mauve, and yellow regions A, B, C, D, E, and F, respectively. The colors give the following complete grid description, saved as `image_grid.txt`:

```
GRID 6 6
ZONES A B C D E F

A A A B B B
A A A C C B
A A A A D B
A A D D D D
```

```
E E E E D D
E E E F F F
```

Each of the six final lines describes one row, from top to bottom; each label identifies the region of one cell, from left to right. Thus the pink region is $R_C = \{(2,4),(2,5)\}$, whereas the green region includes cells in four different rows. The file records regions, not cat positions. Let $R_A, \ldots, R_F$ denote the sets of cells carrying the corresponding labels. These six nonempty sets partition all 36 cells. In formulas indexed by $r = 1, \ldots, K$, we identify $R_1 = R_A, \ldots, R_6 = R_F$.

For every cell (i, j), with row i and column j numbered from 1, introduce a Boolean variable $x_{i,j}$. It is true exactly when a cat is in that cell. A negative literal $\neg x_{i,j}$ means that the cell is empty. DIMACS represents each variable with an integer:

$$\mathrm{id}(i,j) = (i-1)M + j = (i-1)6 + j.$$

The first term skips the $i-1$ preceding rows of six cells; adding j selects the cell in row i. For example, the cat in pink cell $(2,4)$ corresponds to $\mathrm{id}(2,4) = (2-1)6 + 4 = 10$, so its variable is $x_{2,4}$ and its DIMACS positive literal is `10`. The yellow cat in $(6,5)$ has variable number $(6-1)6 + 5 = 35$. All 36 cells have distinct IDs from 1 to 36:

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
$i=1$	1	2	3	4	5	6
$i=2$	7	8	9	10	11	12
$i=3$	13	14	15	16	17	18
$i=4$	19	20	21	22	23	24
$i=5$	25	26	27	28	29	30
$i=6$	31	32	33	34	35	36

.

Table 1 lists every variable belonging to each region. It is equivalent to the colored grid because the preceding formula converts every listed ID back to a unique cell.

Region	Cell-variable IDs	$s_r = \lvert R_r \rvert$
A (green)	1, 2, 3, 7, 8, 9, 13, 14, 15, 16, 19, 20	12
B (turquoise)	4, 5, 6, 12, 18	5
C (pink)	10, 11	2
D (blue)	17, 21, 22, 23, 24, 29, 30	7
E (mauve)	25, 26, 27, 28, 31, 32, 33	7
F (yellow)	34, 35, 36	3

Table 1: Variable numbers of the cells of each region of the board of Figure 1, and size s_r of each region.

The six cats in Figure 1 occupy $(1,2)$, $(2,4)$, $(3,6)$, $(4,3)$, $(5,1)$, and $(6,5)$, with IDs 2, 10, 18, 21, 25, and 35. The corresponding satisfying assignment sets exactly

$$T = \{x_{1,2}, x_{2,4}, x_{3,6}, x_{4,3}, x_{5,1}, x_{6,5}\}$$

to true and all other cell variables to false. The six cats occupy different rows, columns, and regions. Their column sequence $2, 4, 6, 3, 1, 5$ has gaps greater than one between consecutive rows, so none touch diagonally.

1.2 From an expression to DIMACS

A *literal* is a variable or its negation. Literals joined by $\vee$ (OR) form a *clause*; clauses joined by $\wedge$ (AND) form a formula in *conjunctive normal form* (CNF). Every clause must be true. The symbols $\bigvee$ and $\bigwedge$ repeat OR and AND over the indices written below them. A comma in an index lists simultaneous conditions; it is not itself a logical operator.

The pink region has only the cells $(2,4)$ and $(2,5)$. Requiring exactly one cat there gives the small CNF fragment

$$\underbrace{(x_{2,4} \vee x_{2,5})}_{\text{at least one}} \wedge \underbrace{(\neg x_{2,4} \vee \neg x_{2,5})}_{\text{at most one}}.$$

Their IDs are 10 and 11. As a standalone two-clause fragment using the board's 36 variable numbers, the expression becomes

```
p cnf 36 2
10 11 0
-10 -11 0
```

In general, the DIMACS header has the form `p cnf V C`: V is the number of Boolean *variables*, and C is the number of clauses that follow. There is one variable per cell, so the board has $V = NM = 6{\cdot}6 = 36$. The two clauses above are only a fragment of the complete formula, hence $C = 2$ in this example. Each OR-clause becomes one line, and placing both lines in the file represents AND. A positive number denotes a variable and a negative number its negation. Zero ends a clause; it is a delimiter, not variable zero or the Boolean value false.

The complete board formula contains additional clauses for all four rules, each written in the same DIMACS format.

1.3 The four rules as clauses

1.3.1 At most one cat in each row

For distinct cells (i,j) and (i,k) in the same row, at least one must be empty. The clause $\neg x_{i,j} \vee \neg x_{i,k}$ is false precisely when both contain cats. We write it for every pair of columns $j < k$ in every row i:

$$F_{\text{rows}} = \bigwedge_{1 \le i \le N,\ 1 \le j < k \le M} \underbrace{(\neg x_{i,j} \vee \neg x_{i,k})}_{\text{not both cats in row } i}.$$

For example, in the first row, cells $(1,2)$ and $(1,3)$ have IDs 2 and 3. Their clause becomes `-2 -3 0`. The cat at $(1,2)$ makes $x_{1,2}$ true, so the clause requires $x_{1,3}$ to be false. We need a clause for every pair, since excluding only this pair would still allow cats at other two cells of the row. The condition $j < k$ visits each unordered pair once and never pairs a cell with itself. These clauses allow zero or one cat in a row, exactly as the rule states.

1.3.2 At most one cat in each column

Two cells in one column have the same j and different row numbers $i < k$. Applying the same pairwise exclusion gives

$$F_{\text{columns}} = \bigwedge_{1 \le j \le M,\ 1 \le i < k \le N} \underbrace{(\neg x_{i,j} \vee \neg x_{k,j})}_{\text{not both cats in column } j}.$$

In column 4, the cat at $(2,4)$ has ID 10, while yellow cell $(6,4)$ has ID 34. The clause $\neg x_{2,4} \vee \neg x_{6,4}$ becomes `-10 -34 0`. Since the cat at $(2,4)$ makes the first literal false, $(6,4)$ must remain empty. The rule covers any two cells in a column, even if they are not adjacent.

1.3.3 Exactly one cat in each region

For a region R_r, one positive clause requires at least one cat. Negative clauses for all pairs of distinct cells prevent a second cat. Both conditions must hold for every region:

$$F_{\text{regions}} = \bigwedge_{1 \le r \le K} \left[\underbrace{\left(\bigvee_{u \in R_r} x_u \right)}_{\text{at least one cat}} \wedge \underbrace{\left(\bigwedge_{\{u,v\} \in \binom{R_r}{2}} (\neg x_u \vee \neg x_v) \right)}_{\text{at most one cat}} \right].$$

Here u and v are cells, x_u is the variable of cell u, and $\binom{R_r}{2}$ denotes all unordered pairs of different cells in that region. The inner $\bigwedge$ makes one negative clause per pair; the outer $\bigwedge$ repeats the entire requirement for A through F. The pink region C gives exactly the two DIMACS lines shown above: `10 11 0` and `-10 -11 0`. The positive line rules out an empty pink region; the negative line rules out two pink cats. The cat at $(2, 4)$ satisfies the first and forces $(2, 5)$ to be empty through the second. A larger region, such as yellow F with IDs 34, 35, and 36, has the positive line `34 35 36 0` and three pair lines `-34 -35 0`, `-34 -36 0`, and `-35 -36 0`.

1.3.4 No two cats in adjacent cells

The game's eight-neighbor rule includes horizontal, vertical, and diagonal contact. The row clauses already exclude every horizontal pair, and the column clauses exclude every vertical pair. To cover diagonal contact, consider each 2×2 block and forbid its two diagonals:

$$F_{\text{diagonals}} = \bigwedge_{1 \le i < N,\, 1 \le j < M} \left[\underbrace{(\neg x_{i,j} \vee \neg x_{i+1,j+1})}_{\text{first diagonal pair}} \wedge \underbrace{(\neg x_{i,j+1} \vee \neg x_{i+1,j})}_{\text{second diagonal pair}} \right].$$

For the block comprising rows 1–2 and columns 3–4, the first diagonal pair has IDs 3 and 10, giving `-3 -10 0`; the second diagonal pair has IDs 4 and 9, giving `-4 -9 0`. Because the cat at $(2, 4)$ makes $x_{2,4}$ true, the first clause forbids a cat at $(1, 3)$. Every diagonally touching pair belongs to such a block, so iterating over all blocks covers the entire grid without extra clauses for horizontal or vertical adjacency.

The complete formula is

$$F = F_{\text{rows}} \wedge F_{\text{columns}} \wedge F_{\text{regions}} \wedge F_{\text{diagonals}}.$$

For a general $N \times M$ board, rows and columns are constrained by *at most* one, not necessarily exactly one. On this board, $N = M = K = 6$ and the six region clauses require six cats. Since six rows and six columns can hold at most six cats, each must in fact contain exactly one.

1.4 From a grid text file to a CNF file

The converter (`meowdoku_to_dimacs.py`) turns a grid file into a DIMACS CNF file in three phases: it reads and checks the grid, it generates the clauses of the four rules, and it writes them out. Procedure 1 uses the notation of the previous sections: $\text{reg}(i, j)$ is the region label of cell (i, j), and a clause is written as a set of DIMACS literals, that is, nonzero integers whose sign is the polarity.

Procedure 1: Grid file to DIMACS CNF

Input: a grid file. **Output:** a DIMACS CNF file.
ignore blank lines and comment lines (those starting with `c`)
read N and M from the line `GRID` N M and the region names from the line `ZONES`
read the next N lines, each made of M labels; $\text{reg}(i, j) \leftarrow$ the label in row i, column j
if the names are not distinct, the numbers of rows or labels are wrong, or the labels used are not exactly the declared names **then** reject the file
$\text{id}(i, j) \leftarrow (i - 1)M + j$; $F \leftarrow$ empty sequence of clauses
for each row i **and each** pair of columns $j < k$
    append $\{-\text{id}(i, j), -\text{id}(i, k)\}$ to F
for each column j **and each** pair of rows $i < k$
    append $\{-\text{id}(i, j), -\text{id}(k, j)\}$ to F
for each region r
    $S_r \leftarrow \{\text{id}(i, j) \mid \text{reg}(i, j) = r\}$
    append S_r itself to F
    for each pair $u < v$ in S_r: append $\{-u, -v\}$ to F
for each 2×2 block, that is $1 \le i < N$ and $1 \le j < M$
    append $\{-\text{id}(i, j), -\text{id}(i + 1, j + 1)\}$ and $\{-\text{id}(i, j + 1), -\text{id}(i + 1, j)\}$ to F
write the line `p cnf` NM $|F|$
for each clause of F, in order: write its literals separated by spaces, followed by `0`

In Procedure 1, lines 1–4 read and validate the input. Reading ignores any line that begins with `c`, as the DIMACS and grid formats prescribe, so a region name must not be a lowercase `c`. The check on line 4 ensures that every declared region has at least one cell and that no cell has an undeclared label. The procedure does not require $K = N$ or $K = M$, nor that regions be connected: the encoding uses only the fact that the regions partition the cells. Line 5 gives every cell its variable number, so the formula has $V = NM$ variables. Lines 6–15 emit the clauses of the four rules in the order rows, columns, regions, diagonals. Lines 16–17 write the header, where $V = NM$ even if a variable occurs in few clauses, and then the clauses.

Table 2 gives the number of clauses emitted by each rule.

Rule	Number of clauses	Figure 1
Rows	$N\binom{M}{2}$	$6 \cdot 15 = 90$
Columns	$M\binom{N}{2}$	$6 \cdot 15 = 90$
Regions	$\sum_{r=1}^{K}\left(1 + \binom{s_r}{2}\right)$	$67 + 11 + 2 + 22 + 22 + 4 = 128$
Diagonals	$2(N-1)(M-1)$	$2 \cdot 5 \cdot 5 = 50$
Total C		358

Table 2: Number of clauses emitted by each rule, in general and for the board of Figure 1, whose region sizes are $(12, 5, 2, 7, 7, 3)$.

No clause is added for horizontal or vertical contact: two cells sharing an edge lie in the same row or the same column, so the first two loops already forbid them. Clauses are not deduplicated. For example $\{-2, -3\}$ is produced by the row loop and again by the loop of region A, since cells 2 and 3 share both a row and a region; the duplicate is harmless. The output size is dominated by the pairwise clauses, $O(NM^2 + MN^2 + \sum_r s_r^2)$: a generated 50×50 board (seed 1) already gives 196 650 clauses. For the board of Figure 1, the file starts with

```
p cnf 36 358
-1 -2 0
-1 -3 0
```

1.5 Generating solvable grids

The generator (`generate_grid.py`) handles the square case $N = M = K = n$ allowed by the assignment. Its method is to *plant* a solution first and to draw the regions afterwards, so that the board is satisfiable by construction. It takes the size n and an optional seed for the pseudo-random generator, which makes a run reproducible.

Stage 1: planting the cats. The planted solution is a sequence $c_1, \ldots, c_n$, where c_i is the column of the cat in row i. It must satisfy two conditions: the columns are pairwise distinct, so there is one cat per column, and $|c_i - c_{i+1}| \geq 2$, so cats in consecutive rows touch neither vertically nor diagonally. One cat per row holds by construction, and cats two or more rows apart cannot touch. The sequence is found by randomized backtracking over the rows (Procedure 2).

Procedure 2: PlaceCats(i): choose the cat columns of rows $i, \ldots, n$

```
1 if i > n then return success
2 L ← the columns c ∈ {1, ..., n} not used by rows 1, ..., i − 1, with |c − c_{i−1}| ≥ 2 when i > 1
3 shuffle L randomly
4 for each c in L
5     c_i ← c
6     if PlaceCats(i + 1) succeeds then return success
7 release c_i; return failure                    (the caller tries its next column)
```

The case $n = 1$ is solved by $c_1 = 1$. For $n = 2$ and $n = 3$ no valid sequence exists, and the generator reports an error. For $n \geq 4$ a valid sequence always exists, for example the even columns in increasing order followed by the odd ones ($2, 4, 1, 3$ for $n = 4$), so the search always succeeds. In our tests it takes a few milliseconds even for $n = 50$.

Stage 2: growing the regions. The n regions are named $R_1, \ldots, R_n$, region R_i being the one of the cat in row i. The regions are grown from the cats by a multi-source breadth-first search (Procedure 3).

Procedure 3: Growing the regions from the planted cats

```
1 for i = 1, ..., n: reg(i, c_i) ← R_i                    (the cats are the seeds)
2 Q ← queue containing the cells (i, c_i) in random order
3 while Q is not empty
4     remove the first cell (i, j) of Q
5     for each of its four edge-neighbours (i', j'), in random order
6         if (i', j') lies in the grid and has no label yet then
7             reg(i', j') ← reg(i, j);  add (i', j') at the end of Q
8 write the grid file; print the planted solution R_i = (i, c_i)
```

Every cell ends up labeled: if some cell were not, a path of edge-adjacent cells from a seed to it would contain a first unlabeled cell, which is next to a labeled one and would have been labeled when that neighbour was removed from the queue.

The result is a valid board. A label is given only to a cell that has none and is never changed, so each cell has exactly one region and the regions partition the grid. Each region is non-empty, since it contains its seed, and connected, since every cell joins through an edge to a cell of the same region. Region R_i contains exactly one planted cat: its seed is the only seed it has, and the other cats are seeds of other regions and are never relabeled. Hence the planted placement obeys the four rules, and the formula produced by the converter is satisfiable. Because the queue is first-in first-out, the regions grow in lock-step: each cell is claimed by one of the cats closest to it in Manhattan distance, ties being broken at random. The regions are compact but of unequal sizes (from 5 to 12 cells for $n = 8$ and from 14 to 93 for $n = 50$ with seed 1).

Output and properties. The generated file is in the same format as the input of the converter: comment lines, one of which records the seed, then `GRID n n`, `ZONES R1 ... Rn`, a blank line and the n rows of labels. For $n = 5$ and seed 1 it is

```
c Generated grid with a planted solution
c Random seed: 1
GRID 5 5
ZONES R1 R2 R3 R4 R5

R2 R1 R1 R1 R1
R2 R2 R1 R3 R3
R2 R4 R3 R3 R3
R4 R4 R4 R3 R5
R4 R4 R5 R5 R5
```

with the planted solution $R_1 = (1, 3)$, $R_2 = (2, 1)$, $R_3 = (3, 4)$, $R_4 = (4, 2)$, $R_5 = (5, 5)$. The same seed always gives the same board. The generator guarantees at least one solution, not a unique one, and it does not sample all valid boards uniformly. This board has seven solutions, so the planted one is not the only one: in one run, our solver returned a different valid placement, with cats at $(1, 3)$, $(2, 1)$, $(3, 5)$, $(4, 2)$ and $(5, 4)$.

2 Question 2: Reading DIMACS and implementing DPLL

File	Description
`dimacs_to_pylist.py`	DIMACS reader. `read_dimacs` returns the number of variables and the clauses as lists of signed integers, and checks the header, the variable range, the final `0` of each clause and the declared number of clauses.
`sat.py`	DPLL solver: `simplify`, `unit_literal`, `pure_literal`, `choose_literal` (random choice), `dpll_status` (Algorithm 1) and `dpll_model` (Algorithm 2), with a command-line entry point.
`public-instances/instances/`	The 41 DIMACS files provided by the teachers: interface tests, edge cases and 30 benchmark instances (`public-01` to `public-30`).
`public-instances/expected.json`	Expected SAT or UNSAT status of each instance.
`public-instances/README.md`	The teachers' description of the instances.

2.1 From the DIMACS file to Python data

A formula in conjunctive normal form is represented by a list of clauses, and a clause by a list of nonzero integers with the DIMACS convention: a positive integer k is the variable x_k and $-k$ is its negation. The outer list is read as a conjunction (AND) and each inner list as a disjunction (OR). For the board of Figure 1, whose DIMACS file is produced by the converter of Question 1, the file begins with

```
p cnf 36 358
-1 -2 0
-1 -3 0
```

and is read as $V = 36$ and a list of clauses beginning with `[[-1, -2], [-1, -3], ...]`. The final `0` of each line is not stored, since the end of the inner list already ends the clause. The reader (`read_dimacs`) returns the pair formed by the number of variables and the list of clauses; Procedure 4 describes it.

Procedure 4: DIMACS file to Python data

Input: a DIMACS file. **Output:** the number of variables V and the list F of clauses.
1 $V \leftarrow$ undefined; $F \leftarrow$ empty list; $C \leftarrow$ empty list
2 **for** each line of the file, cut into words
3 **if** the line is empty or its first word is `c` **then** go to the next line
4 **if** V is undefined **then**
5 the line must be `p cnf` N M, with N and M non-negative integers; $V \leftarrow N$; go to the next line
6 **for** each word of the line
7 the word must be an integer x
8 **if** $x = 0$ **then** append C to F; $C \leftarrow$ empty list
9 **else if** $|x| > V$ **then** reject the file
10 **else** append x to C
11 reject the file if there was no header line, if C is not empty, or if $|F| \neq M$
12 **return** (V, F)

The list C holds the clause being read. A clause may extend over several lines: words are accumulated in C until a `0` is met. A file is rejected, with an error message, when the header is missing or malformed, when a literal is outside $1, \ldots, V$, when the last clause lacks its final `0`, or when the number of clauses read differs from the number declared in the header (line 11 of Procedure 4).

The representation must keep two empty cases apart. A file without any clause, such as `edge_no_clause_sat.cnf`, is read as the empty list `[]`: a formula with no clause. A line containing only `0`, as in `edge_empty_clause_unsat.cnf`, is read as an empty inner list, so that file becomes `[[1, 2], [],`

`[-1]]`. Thus `[]` is the absence of clauses, whereas `[[]]` is one clause with no literal. Both files are among the instances provided by the teachers.

2.2 The two versions of DPLL

The solver (`sat.py`) implements Algorithms 1 and 2 of the assignment. They follow the same search and differ only in what they return. Algorithm 1 only tells whether the formula is satisfiable: it answers SAT or UNSAT. Algorithm 2 gives the same answer and, when the formula is satisfiable, also returns a valuation, that is which values make the problem SAT. The solution of a Meowdoku board in Question 3 and the model required in Question 6 come from this valuation. Procedures 5 and 6 are the model-producing version, Algorithm 2; `simplify` computes the formula $F[l \leftarrow 1]$ defined in the assignment, and $\bar{l}$ denotes the literal opposite to l. On line 6 of Procedure 6, the literal l to try first is chosen according to a heuristic: $l = p$ means that the variable p is tried first with the value 1, and $l = \neg p$ with the value 0. In this question the heuristic is H_0, the random choice of the assignment: it draws at random a variable p of F and a value $b \in \{1, 0\}$, and l is p if $b = 1$ and $\neg p$ otherwise. Question 4 uses other heuristics.

Procedure 5: Simplify(F, l): the formula $F[l \leftarrow 1]$

```
1 F' ← empty list
2 for each clause C of F
3     if l ∉ C then append C \ {l̄} to F'
4 return F'
```

Procedure 6: DPLL(F): returns a status and a valuation

```
1  if F contains an empty clause then return (UNSAT, ∅)
2  if F contains no clause then return (SAT, ∅)
3  if F contains a unit clause {l}, or otherwise a pure literal l then
4      (s, v) ← DPLL(Simplify(F, l))
5      if s = SAT then return (SAT, v ∪ {l}) else return (UNSAT, ∅)
6  choose a literal l of F according to the heuristic H_0
7  for each l' in (l, l̄)
8      (s, v) ← DPLL(Simplify(F, l'))
9      if s = SAT then return (SAT, v ∪ {l'})
10 return (UNSAT, ∅)
```

The function `dpll_model` is Procedure 6. The valuation is built while the recursion returns: a call that assigns a literal l adds it to the valuation v returned by the call below it (lines 5 and 9), and a branch that fails returns the empty valuation, so the assignments made in that branch are discarded. The function `dpll_status` is Algorithm 1: the same procedure without the valuation, so it only answers `True` (SAT) or `False` (UNSAT). The program differs from the pseudocode of the assignment only by the following implementation choices.

- A valuation is a dictionary from variable numbers to Boolean values instead of a set of literals, and an unsatisfiable answer is `None`. An empty dictionary is a valid valuation, so the code tests `is None`.
- The unit clause used is the first one of the list. When there is none, the pure literal used is the one of smallest variable number, the positive literal first.
- `simplify` builds a new list of clauses and leaves F unchanged. This is how a failed branch returns to the formula it started from.
- The valuation can omit variables that no remaining clause needs. Any value for them satisfies the original clauses, and the command-line interface gives them the value false.

- Each recursive call makes at least one assignment, so the depth can reach the number of variables. The program raises Python's recursion limit according to the number of variables and runs the search in a thread with a larger stack.

2.3 Validation on the teachers instances

Protocol. The parser read all 41 provided CNF files (5 small interface tests, 6 edge cases and 30 benchmark instances). Each file was then solved three times by each version, `dpll_status` and `dpll_model`, with a limit of 60 seconds per run, which is the limit of the final competition. Three runs were performed since the branching choice is random. Every returned status was compared with `public-instances/expected.json`, and every returned model was checked against every clause of the original formula. Times are reported as the median over the runs that finished. The runs were made eight at a time, in parallel, on the machine described in Section 7, so the times are indicative.

Results. No answer was wrong: every status matched the expected one, and every model satisfied all the clauses. Four instances were not solved in every run: `public-03` and `public-27` in no run, and `public-06` and `public-12` in some runs only. The other 37 instances were solved in all runs by both versions. Among the remaining 19 benchmark instances, the median time is at most 0.5 s in both versions. Table 3 gives the 11 benchmark instances that take longer or are not solved in every run.

Instance	Variables	Clauses	Status	`dpll_model`	`dpll_status`
`public-03`	63	168	UNSAT	0/3, timeout	0/3, timeout
`public-06`	1363	3034	SAT	0/3, timeout	1/3, 0.5 s
`public-08`	100	900	UNSAT	3/3, 1.8 s	3/3, 0.7 s
`public-11`	729	21844	SAT	3/3, 36.8 s	3/3, 13.5 s
`public-12`	522	11636	SAT	1/3, 7.1 s	1/3, 3.3 s
`public-15`	100	850	UNSAT	3/3, 1.9 s	3/3, 2.6 s
`public-18`	459	7054	SAT	3/3, 21.0 s	3/3, 10.7 s
`public-19`	729	22060	SAT	3/3, 2.1 s	3/3, 3.3 s
`public-22`	100	850	UNSAT	3/3, 3.0 s	3/3, 3.0 s
`public-27`	181	3151	SAT	0/3, timeout	0/3, timeout
`public-30`	100	850	UNSAT	3/3, 3.5 s	3/3, 1.5 s

Table 3: Benchmark instances whose median time exceeds 1 s in at least one version, or that are not solved in every run. A cell reads "$k/3$, t": k is the number of the three runs that finished within 60 s, and t is the *median* of the times of these k runs.

2.4 Performance

- *The random choice makes the time unstable.* `public-12` was solved by `dpll_status` in 3.3 s in one run and exceeded 60 s in the other two, and `public-18` took from 10.1 s to 56.8 s over the three runs of `dpll_status`. Differences between the two versions, such as 21 s against 11 s (medians) on `public-18`, are of the same order as this variation, so these measurements do not show that one version is faster than the other. Both use the same search.

- *`public-03` is out of reach for this baseline.* It is unsatisfiable, so the whole search tree must be explored, and the random choice of branches does not help. It has only 63 variables but no run finished within 60 s. The teachers' README states that some instances are noticeably hard for a basic DPLL.

- *`public-06` needs deep recursion.* Both functions are recursive, and each assignment adds one level, so the depth can reach the number of variables. This instance has 1363 variables, more than Python's default limit of 1000 nested calls, and an earlier version of `sat.py` stopped on it with a `RecursionError` after about 0.6 s. With the limit raised, it behaves like `public-12`: when

the random choices are favorable it is solved quickly (once in six runs, in 0.5 s), and otherwise it exceeds 60 s.

- *The cost of a step grows with the formula.* `simplify` builds a new copy of the whole remaining formula for every assignment, including forced ones. The two instances with about 22 000 clauses, `public-11` and `public-19`, already need from 2 s to 37 s.

3 Question 3: Producing a Meowdoku solution file

File	Description
`solve_meowdoku.py`	Program of this question: `solve_grid` chains the steps, `cats_from_model` turns a model into cat positions, `check_solution` verifies the placement, and `format_solution` builds the two descriptions of the solution.
`image_grid.txt`	Example input, the board of Figure 1.
`meowdoku_to_dimacs.py`	`read_grid`, `encode_grid` and `write_dimacs` (Question 1).
`dimacs_to_pylist.py`	`read_dimacs` (Question 2).
`sat.py`	`dpll_model`, and `run_deep`, which runs the search with the recursion limit and stack that DPLL needs (Question 2).

3.1 From a grid file to a solution file

The program takes a grid file and a name for the result file. It does not search for the cats itself: it chains the converter of Question 1 and the solver of Question 2, then translates the answer back into the terms of the game.

Procedure 7: Solving a Meowdoku grid file

Input: a grid file. **Output:** a solution file.

1 read the grid: N, M, the region names and $\mathrm{reg}(i, j)$
2 $F \leftarrow$ the clauses of the grid
3 write F to a temporary DIMACS file, then read this file back as a list of clauses
4 $(s, v) \leftarrow \mathrm{DPLL}(F)$
5 **if** $s =$ UNSAT **then** report that the grid has no solution and stop
6 $P \leftarrow \{(\lfloor (t-1)/M \rfloor + 1,\ ((t-1) \bmod M) + 1) \mid \text{variable } t \text{ is true in } v\}$
7 **if** Check(P), Procedure 8, reports a violation **then** report it and stop
8 write the solution file: the text of the grid file; a blank line; for each region r, in the order of `ZONES`, the line r i j where (i, j) is the cat of r; a blank line; then N lines of M symbols, `*` for a cat and `.` elsewhere

In Procedure 7, lines 1 and 2 are the converter of Question 1 (Procedure 1). The assignment asks for the search to go through a DIMACS file, so line 3 writes the formula to a temporary file and reads it back with the reader of Question 2; the file is deleted afterwards, and no CNF file is left in the project. Line 4 is the solver of Question 2 (Procedure 6). A board of size $N \times M$ has NM variables, so the recursion can be deep; the search runs with the same deep-recursion setting as `sat.py`.

Line 6 of Procedure 7 inverts the numbering $\mathrm{id}(i, j) = (i-1)M + j$ of Question 1. For example, on the board of Figure 1, variable $t = 35$ gives $i = \lfloor 34/6 \rfloor + 1 = 6$ and $j = (34 \bmod 6) + 1 = 5$, the yellow cell $(6, 5)$. Only variables that are true are cats; a variable left out of the partial valuation is false, as in Question 2. The region of each cat is read from $\mathrm{reg}(i, j)$, which gives the association between a region and its cat. Check guarantees that there is exactly one cat per region, so line 8 writes exactly one line per region. The output follows Figure 2 of the assignment: a copy of the input, then the cat of each region as a row and a column numbered from 1, then the marked grid.

3.2 Checking the solution

The assignment recommends checking systematically that the placement is legal. The check uses only the grid and the cat positions, never the CNF formula, so it detects an error wherever it comes from: the encoding, the translation of variables into cells, or the solver. If it reports a violation, the program stops with an error and writes no file.

Procedure 8: Check(P): the violations of the rules by the cats P

```
1 for each region r
2     if the number of cats (i, j) of P with reg(i, j) = r is not 1 then report r
3 for each pair of distinct cats (i, j) and (i', j') of P
4     if i = i' then report a shared row
5     if j = j' then report a shared column
6     if |i − i'| ≤ 1 and |j − j'| ≤ 1 then report two cats that touch
7 return the reports
```

Line 6 of Procedure 8 is the eight-neighbour rule of the game: it covers horizontal, vertical and diagonal contact. A placement is legal exactly when no violation is reported.

3.3 Tests

- *Grid of the assignment.* On the grid of Figure 1b of the assignment, the program writes exactly the solution of Figure 2: `A 3 2`, `B 2 5`, `C 1 3`, `D 5 6`, `E 4 4`, `F 6 1`, and the corresponding marked grid.
- *Board of Figure 1.* The solution of this board is unique, so the program must find it. The part of the result file that follows the copy of the input is

```
A 1 2
B 3 6
C 2 4
D 4 3
E 5 1
F 6 5

. * . . . .
. . . * . .
. . . . . *
. . * . . .
* . . . . .
. . . . * .
```

- *Generated boards.* Boards from the generator of Question 1 were solved with seeds 1, 2 and 3, one run each and one at a time, on the machine described in Section 7, and every solution passed the check. Table 4 gives the times, which include about 0.08 s of interpreter start-up.
- *The check itself.* Wrong placements were given to `check_solution`: two cats in one row, two cats touching diagonally, and a cat moved into an occupied column. Each time the corresponding violations were reported, and the legal placement of Figure 1 gave none.
- *A grid without solution.* For a 2×2 grid with regions `A A` and `B B`, the program reports that the grid has no solution, exits with status 1 and writes no file.

n	Variables	Seed 1	Seed 2	Seed 3
4	16	0.07 s	0.07 s	0.07 s
6	36	0.08 s	0.08 s	0.08 s
8	64	0.09 s	0.08 s	0.08 s
10	100	0.11 s	0.15 s	0.10 s
12	144	0.14 s	0.10 s	0.08 s
15	225	0.16 s	1.21 s	5.07 s
20	400	2.58 s	0.41 s	7.00 s

Table 4: Time to solve generated $n \times n$ boards with n regions, for seeds 1, 2 and 3, one run each.

The boards up to $n = 12$ are solved in about 0.1 s. For $n = 15$ and $n = 20$ the time depends strongly on the board and on the random choices of the solver of Question 2, and goes from 0.16 s to 7 s.

4 Question 4: Choosing DPLL branches

File	Description
`heuristics.py`	The branching heuristics: `first_variable` (H1), `literal_scores`, `static` and `dynamic`, which build the static and dynamic versions of H3, H5 and H7, and `FACTORIES`, which names the seven configurations.
`sat.py`	`dpll_h`, the algorithm DPLL$_H$ of the assignment with counters, `Stats` and `make_heuristic`. The random choice H_0 is `choose_literal`, from Question 2.

4.1 The heuristics

Algorithm 3 of the assignment is Algorithm 2 in which the random choice is replaced by a heuristic H that chooses a variable and the value to try first. The function `dpll_h` is this algorithm: it is Procedure 6, in which line 6 chooses the literal according to the heuristic H. We implemented four heuristics, H1, H3, H5 and H7, which give seven configurations once the static and dynamic versions are counted, and we keep H_0, the random choice of Question 2, as the reference. For a formula F and a literal l, $\mathrm{occ}(l, F)$ is the number of occurrences of l in F, $\mathrm{occ}_k(l, F)$ the number of occurrences in the clauses of size k, and $k_{\min}$ the size of the smallest clause of F. A heuristic that chooses a literal l tries $l \leftarrow 1$ first: the variable p with value 1 if $l = p$, and with value 0 if $l = \neg p$.

Heuristic	Score $s(l)$ of a literal l	Versions
H1	no score: the variable of smallest number in F, with value 1	H1
H3	$\mathrm{occ}(l, F)$	H3s, H3d
H5	$\mathrm{occ}_{k_{\min}}(l, F)$	H5s, H5d
H7	$\sum_k \mathrm{occ}_k(l, F)\, 2^{-k}$	H7s, H7d

Table 5: The four heuristics: the score of a literal, and the versions of each heuristic.

For H3, H5 and H7, the literal chosen is the one with the largest score $s(l)$ of Table 5. A tie is broken in favor of the smaller variable number, then of the positive literal. The static version (Procedure 10) computes the scores once, on the initial formula F_0, and ranks all its literals. The dynamic version (Procedure 9) computes them again on the current formula at each choice. Both return a literal whose variable occurs in the current formula.

Procedure 9: Dynamic choice: H3d, H5d and H7d on the formula F

$s \leftarrow$ the score of every literal of F, as defined in Table 5
return the literal with the best score

Procedure 10: Static choice: H3s, H5s and H7s

Before the search, on the initial formula F_0:
$s \leftarrow$ the score of every literal of F_0, as defined in Table 5
$R \leftarrow$ the literals of F_0, from the best score to the worst
At each choice, on the current formula F:
return the first literal of R whose variable occurs in F

Why these choices. The assignment asks for at least four heuristics, one that selects a literal, one that uses clause sizes and one that is dynamic. H3 is the simplest literal heuristic. H5 and H7 use the sizes of the clauses: a short clause is the one that becomes a unit clause first, and unit propagation is the cheapest way to cut the search, so H5 looks only at the shortest clauses and H7 weights each clause by 2^{-k}, which favors the short ones without ignoring the others. H1 costs nothing to compute, which shows what a computed heuristic must gain over a trivial rule. Having both versions of H3, H5 and H7 allows us to measure, in Question 5, what the recomputation brings and what it costs.

4.2 Validation

Each of the eight configurations was run on the 11 small instances of the teachers, which include the edge cases, that is 88 runs, and on the 30 benchmark instances and the 18 Meowdoku formulas of Question 5. Every answer had the expected status, and every model satisfied all the clauses of the formula; no answer was wrong. These are the same test data as in Question 2, so the heuristics agree with the solver of Question 2 on all of them. An instance that is not solved within the limit of 60 s gives no answer, and Question 5 counts it as unsolved.

5 Question 5: Measuring heuristic performance

File	Description
`benchmark.py`	Measurement program: `generate_instances` writes the Meowdoku instances, `run_once` solves one instance with one heuristic and returns the counters, and `run_all` runs every heuristic on every instance, each run in its own process.
`sat.py`	`Stats`, the counters filled by `dpll_h`: decisions, backtracks and time spent choosing branches.
`heuristics.py`	The heuristics compared (Question 4).
`meowdoku-instances/`	The 18 Meowdoku formulas generated for the measurements, and `expected.json`, which gives their status.
`public-instances/instances/`	The 30 benchmark instances of the teachers, 19 SAT and 11 UNSAT.

5.1 Protocol

Instances. We used the 30 benchmark instances of the teachers, and 18 formulas from Meowdoku boards. Nine are satisfiable: boards of size $n = 10$, 15 and 20, with seeds 1, 2 and 3, produced by the generator of Question 1. Nine are unsatisfiable: boards of size $n = 8$, 9 and 10, with the same seeds, built by Procedure 11.

Procedure 11: Unsatisfiable Meowdoku board of size n and seed s

Input: n and s. **Output:** a board without solution.

1 generate the board of size n with the seed s and its planted cats (Procedures 2 and 3)
2 $E \leftarrow$ the cells without a cat, in reading order: rows from top to bottom, each from left to right
3 **for** each of the first three cells (i, j) of E: $\mathrm{reg}(i, j) \leftarrow X$, a new region name
4 **return** the board with the regions $R_1, \ldots, R_n$ and X

The board returned by Procedure 11 has $n + 1$ regions on n rows. Each region needs exactly one cat, and a row holds at most one, so at most n cats can be placed and $n + 1$ are required: the formula of the board, built by Procedure 1, is unsatisfiable. Three cells are given to the new region, and not one, because a region of a single cell gives a unit clause, and unit propagation then refutes the formula without any search; with three cells the solver must search, and the work grows quickly with n.

Heuristics and criteria. The eight configurations are H_0, the random choice of Question 2, and the seven of Question 4. Each pair of instance and heuristic was run once, in its own process, with a limit of 60 seconds, ten runs at a time on the machine described in Section 7. H_0 is random, so its results are one sample. We measure three criteria: the number of decisions, that is the number of executions of the choice of a branch, which is line 6 of Procedure 6 and line 17 of Algorithm 3 of the assignment; the CPU time of the process; and the share of this time spent computing the heuristic. The number of backtracks, which counts the executions of the second branch (the second iteration of the loop of lines 7 to 9 of Procedure 6, and line 21 of Algorithm 3), is, in total, at least 97% of the number of decisions for every heuristic and equal to it on unsatisfiable instances, so it adds nothing to the number of decisions and is not reported. The counters and the timer are only read in the code of `dpll_h`: they do not change the answer of the search, and they do not appear in the pseudocode. Every answer was checked: the status against the expected one, and every model against all the clauses. None of the 384 runs gave a wrong answer.

5.2 Results

	Public SAT (19)	Public UNSAT (11)	Meowdoku SAT (9)	Meowdoku UNSAT (9)	Total (48)
H_0	17	10	9	9	45
H1	18	10	8	9	45
H3s	18	10	7	9	44
H3d	18	10	9	9	46
H5s	18	10	8	9	45
H5d	19	10	9	9	**47**
H7s	17	10	7	9	43
H7d	17	10	9	9	45

Table 6: Number of instances solved within 60 s, out of the number given in parentheses for each group.

Table 6 gives the number of instances solved within 60 s. Instance `public-03` was not solved by any heuristic. The other unsolved instances are `public-12` (by H_0, H1, H3s, H5s and H7s), `public-06` (H_0, H7s and H7d), `public-11` (H3d and H7d), and the boards `sat-n20-s1` (H3s, H5s and H7s) and `sat-n20-s2` (H1, H3s and H7s).

The comparison of the effort is made on the instances solved by all the heuristics, which are 16 of the 19 public SAT instances, 10 of 11 public UNSAT, 7 of 9 Meowdoku SAT and all 9 Meowdoku UNSAT. Table 7 gives, for each group, the total number of decisions, in thousands, and the total CPU time in seconds. Its last column is the share of the CPU time spent computing the heuristic, over the four groups.

	Public SAT		Public UNSAT		Meowdoku SAT		Meowdoku UNSAT		Heuristic
	Dec.	CPU	Dec.	CPU	Dec.	CPU	Dec.	CPU	share
H_0	13.1k	70.0 s	9.4k	15.3 s	3.1k	2.1 s	24.1k	15.3 s	1.8%
H1	6.9k	66.4 s	5.2k	7.2 s	39.1k	29.0 s	1.4k	1.5 s	2.0%
H3s	24.1k	31.9 s	3.4k	5.0 s	33.2k	19.4 s	33.0k	15.5 s	3.3%
H3d	28.0k	50.9 s	2.3k	3.4 s	2.4k	3.4 s	48.4k	17.2 s	8.7%
H5s	6.8k	37.0 s	1.2k	0.5 s	33.2k	17.6 s	33.0k	18.5 s	2.9%
H5d	57.9k	77.3 s	1.0k	0.5 s	2.4k	3.5 s	48.4k	18.8 s	10.8%
H7s	5.3k	29.0 s	1.2k	0.6 s	33.2k	16.4 s	33.0k	15.2 s	3.0%
H7d	18.4k	62.7 s	1.0k	0.5 s	2.4k	3.8 s	48.4k	18.4 s	9.7%

Table 7: Total number of decisions (in thousands) and total CPU time on the instances solved by all the heuristics, for each group, and share of the CPU time spent computing the heuristic, over the four groups.

5.3 Analysis

Are the results in agreement with our expectations? Partly. We expected the heuristics to beat the random choice, the dynamic versions to beat the static ones, and the heuristics based on clause sizes to beat the simple ones. The first holds on the public UNSAT instances, where H5 and H7 need about 1 000 decisions against 9 400 for H_0, and 0.5 to 0.6 s of CPU time against 15.3 s. It does not hold on the Meowdoku boards. On the satisfiable ones, H_0 solves the seven boards in 2.1 s, faster than every heuristic. On the unsatisfiable ones, every frequency-based heuristic needs more decisions than H_0 (33 000 to 48 400 against 24 100), and only H1 does much better, with 1 400. The second expectation holds on the satisfiable Meowdoku boards, where the dynamic versions need 2 400 decisions against 33 200 for the static ones. It fails on the public SAT instances, where H5d needs 57 900 decisions against 6 800 for H5s, and on the unsatisfiable Meowdoku boards (48 400 against 33 000). The third holds only in part: on the public UNSAT instances H5 and H7 are far better than H1 and H3, and H7s is the fastest version on the public SAT ones, but H3d is faster than H5d and H7d there.

How much does a heuristic improve the initial DPLL? The gain depends on the family of instances. On the public UNSAT instances, H5 and H7 divide the CPU time by about 25 to 30 (0.5 to 0.6 s against 15.3 s). On the public SAT instances, H7s takes 29.0 s against 70.0 s for H_0. On the Meowdoku boards there is no gain in this run: H_0 is the fastest on the satisfiable ones and similar to the others on the unsatisfiable ones, except for H1 (1.5 s against 15.3 s). In the number of instances solved, the gain is small: 47 for H5d against 45 for H_0, which does not solve `public-06` and `public-12`. H_0 is a single random run, whose figures vary from one run to another, so these differences are indicative.

Does the dynamic recomputation pay for its cost? The heuristic accounts for 9 to 11% of the CPU time in the dynamic versions, against about 3% in the static ones. The recomputation pays on the satisfiable Meowdoku boards: the dynamic versions need 2 400 decisions against 33 200 and are four to six times faster than the static ones (3.4 to 3.8 s against 16.4 to 19.4 s). They also solve all nine boards, which no static version does, and `public-12`, which neither the static versions nor H_0 solve, so they are more robust. It does not pay on the public SAT instances, where H3d, H5d and H7d are slower than their static versions (50.9, 77.3 and 62.7 s against 31.9, 37.0 and 29.0 s), nor on the UNSAT instances, where the times are about equal. No heuristic is the best on all families. By the number of instances solved, which is the first criterion of the final ranking, the best is H5d.

6 Question 6: Solver interface and optimized mode

File	Description
`sat.py`	Entry point of the solver. `main` reads the options, `run_deep` runs the search, and `solve_optimized` is the optimized mode: `dpll_h` with the heuristic named by `OPTIMIZED_HEURISTIC`.
`heuristics.py`	The heuristics, one of which is used by the optimized mode.
`benchmark.py`	Its command `cli` runs `sat.py` on the public instances in both modes and checks the answers.
`public-instances/instances/`	The 41 instances of the teachers.
`public-instances/expected.json`	Expected status of each instance.

6.1 Interface

The program is run as `python3 sat.py instance.cnf`, with the options `--mode dpll` (the default), `--mode optimized`, and `--output-model`. The mode `dpll` is the solver of Question 2. The mode `optimized` is described below. Both read a DIMACS file, print the same format, and give the same status on every instance.

- For a satisfiable formula the program prints `s SATISFIABLE`, and for an unsatisfiable one `s UNSATISFIABLE`.
- With `--output-model` and a satisfiable formula, it prints a second line that starts with `v`, lists every variable from 1 to V with its sign, positive for true and negative for false, and ends with `0`. A variable that the search left without a value is given the value false. No model is printed for an unsatisfiable formula.
- The standard output contains only these lines. A file that cannot be read is reported on the standard error.

6.2 The optimized mode

The optimized mode is the algorithm DPLL_H of Question 4, that is Procedure 6 in which line 6 chooses the literal according to the heuristic H5d, the dynamic version of the most frequent literal in the smallest clauses. We chose it from the measurements of Question 5: it solved the most instances, 47 of 48, and the number of instances solved is the first criterion of the final ranking. H5d is deterministic, so a run always gives the same result. This is the only difference with the mode `dpll`: the propagation and the data structures are those of Question 2, and the optimized mode is not faster than it by construction. The heuristic is changed by editing one constant, `OPTIMIZED_HEURISTIC`.

6.3 Checks

Both modes were run through the command line on the 41 public instances, one run each, with a limit of 60 s, eight at a time, on the machine described in Section 7. An answer is correct, as in the competition, if its status is the expected one and, for a satisfiable formula, if the model satisfies every clause.

Mode	Correct answers (of 41)	Total time of these answers
`dpll`	39	51.8 s
`optimized`	40	134.7 s

Table 8: Answers of the two modes on the 41 public instances: number of correct answers and total time of these answers.

Table 8 gives the results. No answer was wrong in either mode. The mode `dpll` did not solve `public-03` and `public-27` within 60 s. The mode `optimized` solved `public-27`, in 40.7 s, but not `public-03`, which no heuristic solved in Question 5. On the 39 instances solved by both, the optimized mode takes 94.0 s against 51.8 s, mostly on `public-06` (35.1 s against 0.6 s) and `public-12` (25.9 s against 12.4 s), which the mode `dpll` solved quickly in this run thanks to its random choices. These times depend on those choices, as seen in Question 2, so the comparison is indicative; the mode `optimized` is reproducible. The ranking counts the instances solved first, and the total time only second, which is the reason for the choice of H5d.

7 Experimental setup

All the measurements of this report were made on the computer described in Table 9.

Component	Configuration
Computer	Acer Nitro ANV16-42-R3JW
Processor	AMD Ryzen 7 260 with Radeon 780M Graphics, x86_64
Cores	1 socket, 8 physical cores, 2 threads per core, 16 logical CPUs
Cache	L1 data 256 KiB and L1 instruction 256 KiB (8 instances each), L2 8 MiB (8 instances), L3 16 MiB
CPU frequency policy	scaling governor `powersave`
Memory	14 GiB of RAM (15 126 480 kB), 4 GiB of swap
Memory modules	2 modules of 8 GB (16 GB), DDR5, speed 5600 MT/s
Graphics	NVIDIA GeForce RTX 5060 (dedicated, 8 GB GDDR7) and the integrated AMD Radeon 780M; the solver uses neither
Storage	NVMe drive; the file system holding the project has 128 GB
Operating system	Ubuntu 26.04.1 LTS, Linux kernel 7.0.0-38-generic
Python	CPython 3.14.4

Table 9: Configuration of the computer used for the measurements.

The measurements of Question 2, Question 5 and Question 6 ran several solver processes at the same time, eight, ten and eight respectively, on the 16 logical CPUs; those of Question 3 ran one process at a time. The processes that run together share the caches and the memory bandwidth, so the times are indicative.

This computer is not the one of the final competition. The assignment specifies an Intel Core Ultra 7 155H, Ubuntu 22.04 and Python 3.12 in a Docker container limited to 4 CPUs and 8 GiB of memory, with a limit of 60 seconds per instance. The times of this report cannot be compared directly with those of the competition.